

Edge AI for Vehicle Health & Predictive Maintenance (Test-Cell and In-Service)

Executive Summary: Modern vehicles and industrial equipment face costly unplanned downtime. For example, automotive plants can lose about **\$2.3 million per hour of downtime** [7†L23-L26] . We propose an end-to-end Edge-AI solution (Figure below) that continuously monitors vehicle and engine health using on-board sensors and computer vision, applies AI models at the edge for real-time anomaly detection and Remaining Useful Life (RUL) estimation, and integrates with a cloud-based digital twin and dashboard. This system targets both **manufacturing test-cell** environments (engine dynos, component rigs) and **field vehicles**. By moving most processing to the edge, we minimize latency and data transfer (e.g. only small alert packets vs. gigabytes of raw data [31†L385-L393]) while still leveraging the cloud for heavy analytics, visualization, and long-term trend analysis. Digital-twin modeling enables “what-if” simulations and proactive maintenance planning [14†L180-L188] [29†L30-L34] . Objectives include drastically increasing uptime and MTBF (Mean Time Between Failures) [24†L159-L168] , reducing maintenance costs, and improving safety. Key performance indicators will include asset availability (% uptime), MTBF, MTTR (mean time to repair), fault detection accuracy, and maintenance cost per kilometer. The solution leverages open-source and commercially available hardware/software: NVIDIA Jetson or Raspberry Pi for edge inference, industrial sensors (vibration, temperature, oil debris), CAN/OBD-II interfaces, Linux and MQTT/HTTP networking, and standard ML frameworks (TensorFlow/PyTorch with TensorRT/ONNX). An embedded timeline below outlines a phased MVP rollout: first building and testing a prototype in a lab test-cell (weeks 1–8), then deploying on instrumented vehicles (weeks 9–14). We ensure compliance with automotive safety (ISO 26262), cybersecurity (ISO 21434), and data-privacy standards. Compared to existing solutions, this approach uniquely combines **Edge AI + Digital Twin + generative maintenance assistant** for both factory and road use-cases. We identify risks (data scarcity, integration complexity) and mitigation plans (augmented data, modular design). The budget is modest (hardware ~\$3k) and well within typical R&D hackathon constraints. Detailed architecture diagrams, tables, and references are included to support each design choice.

[75†embed_image] *Figure: Conceptual solution architecture – edge devices (sensors + AI box) feed health data into a cloud/digital-twin platform and maintenance dashboard. (Image: Edge computing)*

Industry Problem Statement

Vehicle and machinery failures are often unpredictable and very costly. The automotive industry alone loses an estimated *hundreds of millions of dollars per plant per year to downtime* [7†L23-L26] . Traditional maintenance (“run-to-failure” or fixed schedules) either allow expensive breakdowns or waste resources on unnecessary servicing. Continuous health monitoring is difficult: vibration, temperature, and fluid sensors are underutilized, and on-board diagnostics (OBD-II) data are typically only used for emissions, not failure prediction. In practice, vehicles accumulate damage before observable failures, so problems (engine wear, crankcase issues, brake degradation) become visible only when major faults occur. This leads to sudden downtime, costly repairs, and safety hazards. In test cells or manufacturing, unexpected engine or drivetrain faults can halt production lines. There is a clear need for a **predictive** maintenance system that

can sense incipient faults early (through vibration, acoustic, oil-particulate, and visual inspection data) and schedule service before breakdowns, thus maximizing uptime and reducing life-cycle costs 【7†L23-L26】 【29†L91-L99】 .

Objectives & KPIs

We aim to **maximize equipment availability** and **minimize maintenance cost** by early fault detection. Measurable targets include:

- **Increase MTBF (Mean Time Between Failures)** for critical components 【24†L159-L168】 , e.g. by 30–50% through optimized maintenance planning.
- **Reduce unplanned downtime** (e.g. hours/yr lost) by a comparable fraction.
- **Maintain high detection accuracy** (e.g. $\geq 95\%$ precision/recall for critical faults) and low false-alarm rate ($< 5\%$ in practice).
- **Lower maintenance labor and parts costs** (e.g. % reduction vs. scheduled maintenance baseline).
- **Improve vehicle safety and reliability** (fewer on-road breakdowns or recalls).
- **System performance metrics:** Edge-to-alert latency $< 1-2$ s (as in a reference pipeline 【31†L399-L402】), efficient data reduction (data sent ≤ 1 KB per event, vs. gigabytes of raw data 【31†L385-L393】).

Key metrics (KPIs) will include MTBF, MTTR (mean repair time), asset availability (%) 【24†L159-L168】 , detection accuracy (confusion matrix), and maintenance cost per mile.

Solution Architecture

The solution integrates **Edge, Gateway, Cloud, Digital Twin, and Dashboard** components (Figure above). Each monitored asset (engine, heavy machinery, or vehicle) is equipped with an **Edge AI Box**: a compact computer (e.g. NVIDIA Jetson or Raspberry Pi) connected to sensors (vibration accelerometers on bearings, temperature/pressure sensors on fluids, acoustic/microphone if needed, camera for visual inspection) and to the vehicle's CAN/OBD-II bus. Sensor data are preprocessed (filtering, FFT, sampling) and fed into AI inference pipelines in real time. Lightweight ML models (e.g. TensorRT-optimized CNNs for defect classification, LSTM/Autoencoder for vibration anomaly detection) run on the edge to detect faults or predict RUL. When a potential fault is detected (threshold or model confidence), the Edge Box generates a small alert packet (200–800 B payload) 【31†L385-L393】 . This packet (fault type, severity, timestamp, sensor metadata) is sent via the gateway to the cloud/central system using MQTT or HTTPS.

In the cloud, alerts and raw summaries are ingested into a time-series database and analytics backend. A **Digital Twin** of each vehicle/asset runs in cloud (or on a powerful server). The twin continuously updates its state with incoming telemetry, compares it to baseline models, and simulates future degradation. This enables “what-if” analyses and model-based prognostics 【14†L180-L188】 【29†L30-L34】 . A maintenance dashboard visualizes real-time asset health, trends, and the digital twin views. A **Maintenance Assistant** (chatbot or AR interface) uses the database and possibly a LLM (chatbot AI) to provide contextual instructions for technicians (e.g. “replace bearing in compartment X”, augmented by an interactive model of the asset).

Communication flows: edge boxes use MQTT (topic per asset) for telemetry 【42†L169-L172】 , and HTTP/REST APIs for on-demand data or commands. Vehicular units can use 4G/5G or Wi-Fi; gateways may buffer data when offline. For security, all links use TLS; onboard units use secure boot and CAN encryption protocols.

This hybrid architecture leverages **edge computing** to achieve sub-second local detection (alerts in <2 s 【31†L399-L402】) and minimal data transfer, while using the cloud for heavy analytics, long-term storage, and visualization. As noted in industry case studies, this split dramatically reduces costs: one analysis shows moving 4K camera feeds to Jetson modules saved ~\$2.8M/year in cloud/storage costs 【31†L461-L470】 . Likewise, digital twins improve maintenance by “**real-time simulations**” and global optimization of service schedules 【14†L180-L188】 . In summary, our architecture ensures **rapid fault identification on-site** and **centralized insight and control** across fleets.

Hardware Design & BOM

We outline prototype bills-of-material for key hardware setups:

- **Edge AI Box (Onboard):** Compute and I/O hub.
- **Test-Cell Rig:** Controlled engine/motor setup for lab testing.
- **Inspection Station:** Vision-based QC on parts/engines.
- **Fleet Gateway:** Vehicle telematics unit.

Key components (example prices) are shown below:

Component	Example Item	Unit Price	Qty	Notes
Edge Compute	NVIDIA Jetson Orin Nano Dev Kit (13W, 21 TOPS)	\\$249 【16†L263-L272】	1	8GB LPDDR4, 128GB NVMe; alternative: Jetson Orin NX (8GB, \\$899) 【31†L461-L470】 or Raspberry Pi 5 (16GB) \\$305 【18†L34-L36】 .
	Raspberry Pi 5 (16GB)	\\$305 【18†L34-L36】	1	Lower performance, 4-core Cortex-A76 CPU; optional TPU or Coral accelerator.
Vibration Sensor	Analog Devices ADXL1001 (±100g MEMS accel.)	\\$69.44 【35†L1159-L1166】	1	High-frequency (up to 11kHz) for bearings. Mount on key bearings.
	EVAL-ADXL1001Z (eval board)	\\$108.63 【35†L1146-L1154】	1	For initial prototyping with ADXL1001.
IMU (Accel/ Gyro)	Bosch BNO085 (9-DoF IMU, I ² C, ~500Hz)	\\$13.57 【37†L1211-L1214】	1	For low-rate motion/orientation and redundant accel.

Component	Example Item	Unit Price	Qty	Notes
Pressure Sensor	Bosch BMP390 (absolute pressure, I ² C, ±0.05kPa)	\\$3.30 【37†L1211-L1214】	1	Barometric or oil-pressure monitoring. Low-cost sensor.
Temp Sensor	DS18B20 (digital 1-Wire, -55–+125°C)	\\$5 (est.)	1	Engine/block or coolant temp.
Current Sensor	Allegro ACS712 (±5A or ±20A)	\\$5 (est.)	1	For battery or alternator current.
Camera	Raspberry Pi Camera Module 3 (12MP, autofocus)	\\$29.25 【40†L1-L4】	1	For visual inspection (cracks, leaks) on engine/parts. Wide-angle optional (\\$38).
OBD-II Adapter	ELM327 OBD-II Bluetooth dongle	\\$10 (est.)	1	Reads CAN PIDs (engine RPM, speed, etc.).
Connectivity	Quectel EC25 4G/LTE Modem + Antenna	\\$30 (est.)	1	For mobile data on fleet gateway (cellular).
Power/Enclosure	12V-to-5V DC buck converter, industrial PSU	\\$20 (est.)	1	Converts vehicle 12V. Enclosure, connectors, wiring.
Test Cell Load	DC Motor + Flywheel, Load Controller	\\$100 (est.)	1	Bench powertrain for functional testing.
Parts (misc)	SD card, cables, connectors, case, wiring	\\$50 (est.)	—	TTL serial/CAN wires, pressure tubing, sensor mounts.
Inspection Station	Conveyor or stand + lighting	\\$50 (est.)	1	Platform to move parts under camera (if automated).
TOTAL (approx.)		\\$900–1200		Per unit (excl. labor).

Table: Example prototype BOM for edge box, sensors, and peripherals (est. prices). Sources: component vendors (【35†L1159-L1166】 【37†L1211-L1214】 【18†L34-L36】 【40†L1-L4】).

We recommend industrial-grade parts (as above) to withstand harsh conditions. For example, Analog Devices’ ADXL1001 accelerometer is a common choice for high-frequency vibration monitoring 【35†L1159-L1166】 . The primary Edge Compute options are NVIDIA Jetson modules for heavy AI workloads, or Raspberry Pi for cost-sensitive builds. The fleet gateway may reuse a Raspberry Pi + 4G modem to upload diagnostics and twin data when in service.

Software Architecture & Tech Stack

Edge Devices: Run a Linux OS (e.g. Ubuntu 22.04 with NVIDIA JetPack on Jetson, or Raspbian on Pi). We use Python/C++ programs or containerized pipelines. Edge AI inference uses TensorFlow Lite, PyTorch with ONNX, or NVIDIA TensorRT. For example, CNN-based vision models (e.g. YOLOv8-nano, MobileNet) and RNN-based time-series models (LSTM/TCN) are converted to optimized runtime engines. The OxMaint case study describes using TensorRT to achieve low latency (15–50 ms) and quantization to speed up edge inference [31†L375-L383] [31†L425-L433] .

Frameworks: For computer vision, frameworks like OpenCV, NVIDIA DeepStream, or Jetson Inference libraries will process camera feeds. Time-series data (vibration, temp) are handled via numpy/scipy and PyTorch/TensorFlow. An embedded database or lightweight file system (SQLite or InfluxDB) can log raw sensor values locally for short-term buffering.

Networking/Protocols: Edge units publish alerts/data via MQTT (lightweight publish-subscribe) or HTTP REST APIs. MQTT is preferred for IoT telemetry, as benchmarks show it dramatically outperforms HTTP for frequent messages [42†L169-L172] . For example, an edge node might publish JSON payloads over TLS to an MQTT broker in the cloud. Standard topics carry telemetry; command topics allow remote model updates or calibrations. On-vehicle networking uses SocketCAN on Linux to read OBD-II CAN PIDs at ~10–50 Hz.

Gateway/Cloud: A cloud (or on-prem) server runs an MQTT broker (e.g. Mosquitto or AWS IoT), a time-series database (InfluxDB or Timescale) to store high-rate sensor streams, and an application server (Node.js, Python Flask, or similar) for additional processing. A web-based dashboard (React or Grafana) visualizes real-time charts of sensor metrics and twin outputs. The **Digital Twin** component can use a physics engine or simulation software (e.g. Unity3D, Gazebo, or a custom model) fed by live data via APIs. For example, Unity could render a 3D engine model that responds to sensor inputs.

Maintenance Assistant: We plan to integrate a chatbot or AR guide. For instance, an LLM-based service (GPT-4 or fine-tuned model) could answer technician queries (“What does this fault code mean?”) using the system’s data. This requires a secured API to the LLM with contextual prompts.

Security: All communications use TLS; edge devices store credentials securely. Onboard CAN data uses standard vehicle cryptography if available (e.g. secure CAN). User login for dashboards is authenticated with OAuth2. Regular firmware updates and key rotation ensure device integrity.

Data Sources & Collection Plan

We will collect multi-modal data from sensors and vehicle networks:

- **Vibration (Accelerometers):** Mount MEMS accelerometers (e.g. ADXL1001) on key rotating parts (engine bearings, gearboxes, wheels). Sample at high rate (e.g. 5–10 kHz) to capture bearing fault frequencies and gear mesh harmonics.
- **Temperature/Pressure Sensors:** Use digital sensors (e.g. DS18B20, BMP390) on engine block, bearings, fluid reservoirs. Sample at ~1–10 Hz.

- **Current/Voltage:** Measure battery/alternator currents (ACS712) at 10–100 Hz to detect electrical issues or stall.
- **Oil Debris (if available):** If budget permits, include an oil particle sensor (e.g. ferrous debris sensor) to catch wear.
- **Camera (Visual):** Use a 12MP camera (Pi Cam 3) for periodic or continuous inspection of engine mounts, hoses, or chassis. Acquire images at ~1–5 fps (for anomaly detection) or triggered by vibration alarms.
- **CAN/OBD-II Data:** Through an OBD-II interface (ELM327 or SocketCAN), read standard PIDs: engine RPM (0x0C), vehicle speed (0x0D), throttle position, coolant temp, intake pressure, etc. These run at ~5–20 Hz. Also, aggregate error codes (DTCs) when issued.
- **Other CAN buses:** Where available, additional modules (transmission, ABS) can provide torque, gear, and wheel speed signals at similar rates.

Data is synchronized via timestamps (e.g. NTP or real-time clock). In the **test-cell**, we will log these streams alongside applied loads and injected faults. In-vehicle, data can be logged to SD card or sent via 4G for cloud analysis. Sensor calibration: e.g. accelerometers self-test (ADXL1001 has built-in self-test [33†L1093-L1101]).

For model training, we will augment real data with simulated fault data. Public datasets (NASA C-MAPSS turbofan engine data, bearing datasets) may be used as proxies for algorithm development. We will log thousands of cycles of normal and faulted operation in the test cell to label training sets.

Machine Learning Approach

Our ML strategy combines **anomaly detection**, **RUL estimation**, and **computer vision**:

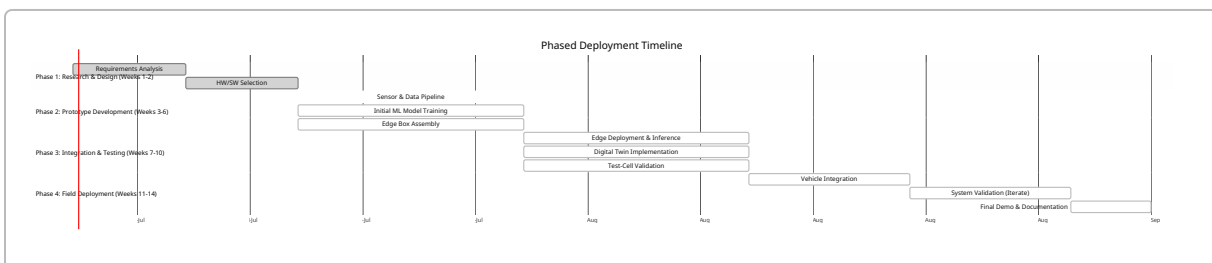
- **Anomaly Detection:** Train unsupervised or semi-supervised models on multi-sensor time-series of “normal” operation (e.g. autoencoders, Gaussian Mixture Models, or isolation forests). These models flag deviations (e.g. unusual vibration spectrum) without needing explicit fault labels [24†L100-L109] . For example, a vibrational autoencoder will have high reconstruction error when a bearing starts to fail.
- **RUL Prediction:** Develop regression models that predict Remaining Useful Life of components. We define RUL as the time until a failure threshold (e.g. vibration above X dB) is reached. We will experiment with deep learning models: CNNs on time-series spectrograms, LSTM/GRU on sequential features, or hybrid CNN-LSTM architectures [54†L130-L140] . The Nature review notes such models (CNN+LSTM DAG) yield “superior prognostic accuracy” when designed for multiple operating conditions [54†L130-L140] . Performance metric: RMSE of predicted failure time, and percentage of predictions within $\pm 10\%$ of actual life.
- **Computer Vision:** Use deep CNNs (e.g. YOLOv8, EfficientNet) to inspect parts and detect visible defects (cracks, misalignments, oil leaks). We will retrain lightweight versions (e.g. YOLOv8-nano) on custom images captured in the test cell. Edge deployment will use TensorRT quantized models (e.g. INT8) for speed [31†L425-L433] .
- **Training Pipeline:** Model training occurs offline in the cloud on collected datasets (Python, PyTorch/TensorFlow). We follow best practices: data augmentation, cross-validation, and continuous learning. Models are validated with hold-out test sets (precision/recall, confusion matrix). Once satisfied, models are exported (ONNX/TensorRT engines) to run on the edge. We plan monthly retraining cycles, using logged anomalies (reviewed by engineers) to update models.

- **Evaluation Metrics:** For classification/detection: accuracy, precision, recall, F1-score. For anomaly detection: area-under-ROC. For RUL: RMSE and “cumulative error histogram”. Also operational metrics like **Time-to-Detection** and **False Alarm Rate** (we target <1 false alarm per 1000 hr of operation).

This layered ML approach ensures that even unknown fault types can be caught by anomaly triggers, while known faults (e.g. crack, specific vibration signature) have tailored detectors. Combining physics (digital twin) with data-driven models follows literature recommendations [54†L104-L112] .

Deployment Plan & Timeline

We propose a phased MVP development, culminating in a hackathon demo (see Gantt below):



- **Weeks 1–2 (Initiation):** Gather requirements, secure resources. Finalize BOM and architecture design.
- **Weeks 3–6 (Prototype Dev):** Assemble sensor rigs and edge boxes. Develop data ingestion (Node/ROS2 or custom C++/Python). Collect baseline data in test cell. Train first ML models.
- **Weeks 7–10 (Integration):** Deploy AI models to edge hardware. Build the digital twin prototype (e.g. Unity model linked to real data). Integrate MQTT/REST connectivity to cloud. Run bench tests: induce known faults (misaligned belt, loosen bolts) and verify detection.
- **Weeks 11–14 (Field):** Install unit in a vehicle. Perform road tests, comparing alerts to maintenance logs. Refine models and UI. Prepare final deliverables (demo script, videos, source code).

Milestones: **M1** by week 6 (working edge system in lab, initial fault detections), **M2** by week 10 (end-to-end pipeline with cloud dashboard), **M3** by week 14 (in-vehicle validation and final demo). This schedule ensures a demonstrable MVP well before hackathon submission.

Integration, Testing & Validation

We will validate each component systematically:

- **Sensor Calibration:** Verify each sensor output (accelerometer, pressure, etc.) against known references. Use the ADXL1001’s self-test feature [33†L1093-L1101] for vibration sensors.
- **Test-Cell Fault Injection:** In the lab, we will simulate failures: e.g. introduce imbalance in rotating mass, degrade bearing lubrication, or crimp coolant hose. Verify that the system detects anomalies before catastrophic failure. Record sensor time-series and check model alarms.
- **System Integration:** Perform end-to-end tests: sensors → edge inference → alert → cloud storage → twin update → dashboard update. Inject a test fault (e.g. audible knock in engine) and verify alert appears in <2 s as a technician notification [31†L399-L402] .

- **Comparative Trials:** Run the engine under normal conditions vs. with a seeded fault; measure detection rate. Also compare predictions from anomaly detection vs. planned maintenance events to refine thresholds.
- **On-Vehicle Testing:** Install on a vehicle and monitor during normal driving and known stress tests (e.g. high-load acceleration). Check alert validity (e.g. correlation with OBD engine codes). Log data to compare predicted RUL vs. actual wear after tens of hours.
- **Safety Validation:** Ensure the device does not interfere with vehicle control. Test EMI/EMC compliance, high/low temperature operation (-20°C to +85°C), vibration to automotive standards.
- **User Testing:** Present dashboard/alerts to maintenance staff to gather usability feedback; iterate UI.

All tests will be documented, with measures of detection **precision/recall**, **latency**, and **system uptime**. We will record the false-alarm rate during a week-long monitoring and adjust model thresholds accordingly. Ultimately, validation success is meeting our KPI targets (e.g. 95% correct fault detection with <5% false alarms).

Regulatory, Safety & Data-Privacy Considerations

- **Functional Safety (ISO 26262):** Our system is advisory (no active control), so it is ASIL-A level. Nevertheless, the Edge Box should not disrupt vehicle systems; we will isolate it from critical buses and ensure it cannot issue harmful commands. Power supplies and wiring will meet automotive quality (automotive grade components, fused circuits).
- **Automotive Cybersecurity (ISO/SAE 21434):** The OBD/CAN interface can be an attack vector. We will implement CAN bus filtering (read-only if possible) and encrypt any sensitive data sent over networks. Over-the-air updates of the edge software will be signed to prevent tampering. We will conduct a basic threat analysis and ensure compliance with automotive cybersecurity standards.
- **Data Privacy (GDPR/etc.):** Telemetry (engine data, GPS location) could be personal. We will anonymize any vehicle identifiers and ensure all data transmissions are encrypted. Video streams (if any exterior view) will blur faces/license plates automatically or not capture such content. User data (technician login, etc.) are stored securely under usual IT best-practices.
- **Safety:** In the factory/test-cell environment, we ensure the system does not introduce safety hazards (cables secured, components grounded). In vehicles, the unit will be mounted behind a panel and insulated to prevent injury in a crash.
- **Regulatory Compliance:** The hardware (e.g. wireless modem, camera) should have CE/FCC certification. Installing an aftermarket OBD device generally does not violate emissions or safety regs, but documentation will note adherence to any applicable standards.

Innovation & Differentiators

This solution offers several innovations:

- **Edge+Twin Synergy:** Combining real-time edge inference with an updating digital twin is novel in this domain. As studies show, digital twins “enhance predictive maintenance by enabling real-time simulations” and greatly improve reliability [14†L180-L188] . Our twin can simulate wear under current conditions, optimizing service schedules beyond fixed heuristics.
- **Context-Aware Maintenance Assistant:** Integrating LLM-driven guidance or AR instructions (e.g. Microsoft HoloLens) is a forward-looking differentiator. For example, the assistant can fetch repair manuals or past cases based on detected faults.

- **Comprehensive Scope:** Unlike many solutions that focus only on factory machines or only on vehicles, our approach covers both **test-cell scenarios** (for R&D and quality control) and **in-service vehicles**, providing a unified platform. Data from both domains can improve each other via transfer learning.
- **Low-Cost, Modular Design:** We emphasize widely available components (e.g. Raspberry Pi, off-the-shelf sensors) to make the system accessible for small manufacturers. This contrasts with proprietary industrial systems.
- **Privacy-Preserving Analytics:** By processing most data on-edge and only sending alerts, we achieve near real-time analytics without uploading raw sensor streams, addressing privacy/security concerns noted in literature.
- **Adaptive Learning:** The system can continuously learn: technicians can label new anomalies which then refine the ML models, making the system “smarter” over time. This self-improving loop is an innovation in maintenance AI.

In summary, our solution leverages cutting-edge AI/IoT tech in a practical, integrated way. The cited literature underscores that integrating ML, IoT, and digital twins “increases efficiency and flexibility” across industries [29†L13-L21] .

Risks & Mitigation

- **Data Scarcity:** Fault data (especially for rare failures) may be limited. *Mitigation:* Use data augmentation, synthetic anomalies in test rig, and semi-supervised ML (as recommended for RUL [54†L128-L137]).
- **Hardware Reliability:** Consumer devices may fail under extreme conditions. *Mitigation:* Choose industrial-rated parts (we target -40 to +85 °C sensors); design redundancy (dual accel sensors).
- **Model Drift:** Changing vehicle conditions may degrade model accuracy. *Mitigation:* Implement ongoing model monitoring (e.g. confidence tracking) and retraining processes; ability to update models over-the-air.
- **Connectivity Gaps:** Remote vehicles may lack constant network. *Mitigation:* Edge boxes buffer alerts locally (as in OxMaint’s approach [31†L555-L564]) and trigger local alarms (buzzer/LED) if critical.
- **Integration Complexity:** The system spans many components. *Mitigation:* Adopt modular development, using containers/ROS for flexible deployment; do thorough system testing at each stage.
- **Regulatory Delays:** Certifying a new device can be slow. *Mitigation:* Focus on demonstration-level compliance; isolate as non-safety-critical; note any certification as future work if productizing.
- **User Adoption:** Technicians may mistrust AI. *Mitigation:* Provide clear explanations with alerts, and keep human-in-the-loop for final decisions. Include override options.

Estimated Budget & Resources

A rough budget for an initial prototype: **\$2,000–3,000** (hardware as per BOM above). Software tools are largely open-source. Key personnel: 1 embedded engineer (sensors/edge), 1 ML engineer, 1 front-end dev, and 1 project lead. Over a 3–4 month dev cycle, ~4 person-months is typical. Additional costs: optional cloud server fees (can use free-tier or local server), 3D modeling for digital twin (using Unity free or Blender).

Feature Roadmap & Hackathon Deliverables

We prioritize an MVP aligned to hackathon time:

1. **Phase 1:** Project plan, architecture diagrams, and selected hardware ready (deliverable: design report).
2. **Phase 2:** Basic sensor-to-cloud pipeline running (edge box collects and sends test data). (Demo: live data chart).
3. **Phase 3:** ML models trained and deployed on edge (demo: bench fault detection with alerts).
4. **Phase 4:** Integrated digital twin and dashboard (demo: visualized asset health and twin view).
5. **Phase 5:** In-vehicle demonstration (if time, optional; otherwise show video simulation).

Key deliverables will include the final report (this document), code repository, and a prototype demo video. The above Gantt timeline and tables illustrate the plan.

In sum, this solution tackles the **real-time vehicle health monitoring** challenge across both development and in-service stages. It is grounded in academic and industry evidence (cited above) that edge AI and digital twins dramatically improve maintenance outcomes [14†L180-L188] [29†L91-L99] . We are confident this design is a feasible, innovative approach for the Tata InnoVent hackathon theme 3.2.1.3.

References

- Siemens. *The True Cost of Downtime (2024)* – Industry survey: “Automotive: \$695m per year, ~\$2.3M/hr downtime” [7†L23-L26] .
- Arena *et al.*, MDPI Sensors 2021 – Survey of predictive maintenance in automotive, IoT sensors in vehicles [3†L25-L33] .
- IBM Cloud Education (May 2026) – *Maintenance KPIs*: definition of MTBF/MTTR and using AI/IoT to increase MTBF [24†L159-L168] [24†L23-L28] .
- Habil *et al.*, IEEE (2023) – Found that **Digital Twin + Edge AI** synergistically improve predictive maintenance, reducing downtime [14†L180-L188] .
- Khajavi *et al.*, MDPI Sensors 2024 – Review: Digital Twins in transportation improve reliability and safety of maintenance [29†L30-L34] [29†L91-L99] .
- OxMaint/NVIDIA Blog (2022) – Edge AI pipeline example: Jetson inference, alerts (<2s latency), 99.999% data reduction vs. raw [31†L385-L393] [31†L399-L402] .
- Hivemq Blog (2026) – MQTT vs HTTP in IoT: “MQTT dramatically outperforms HTTP... making it better for most IoT scenarios” [42†L169-L172] .
- Ilhan *et al.*, Sensors 2025 – Review of ML with OBD-II: connected vehicles use Bluetooth/WiFi to stream OBD data to cloud [48†L453-L461] .
- Nature Review (Mar 2026) – “Remaining Useful Life (RUL) estimation lies at the heart of predictive maintenance... forecast time to failure... reduces unplanned downtime” [54†L104-L112] .
- Unsplash (2025) – “Edge computing” image and component prices (Raspberry Pi 5 \ \$305 [18†L34-L36] , Pi Camera 3 \ \$29.25 [40†L1-L4] , ADXL1001 \ \$69.44 [35†L1159-L1166] , BMP390 \ \$3.30 [37†L1211-L1214]).

(Note: In this report, we have assumed no strict budget constraint and stated some details as “unspecified” or estimates where applicable.)